

Project topic:	Smart Laundry Hamper Level + Washer Detection System
Author:	Joshua Yero

1. Project description

a. General idea of your IoT system

The idea of this IoT system is to build a smart laundry setup that actually helps you stay on top of your chores without thinking about it. The first part is a smart laundry hamper that uses an ultrasonic sensor to automatically measure how full it is. Instead of letting clothes pile up “for another day,” the system checks the hamper constantly and sends real-time height data to the cloud through MQTT. The goal is to nudge you when the hamper starts getting full, almost like having a built-in reminder that keeps you accountable. This is a personal problem as with my busy life I kind of allow my pile to stack up which always ends up with me doing laundry for my entire closet.

The second part of the system uses an accelerometer attached to the washing machine. It monitors vibration and movement to detect when the washer is running and when it has completely stopped. This means you’ll know the moment your laundry is done instead of forgetting it in the washer for hours (which I’m definitely guilty of).

All of this ties together through a cloud dashboard where the user can see both the hamper fullness and the washer status at any time. It creates a simple, practical way to automate a problem I’ve personally had my whole life: waiting too long to wash clothes or forgetting them in the washer. The whole point was to take a basic everyday issue and turn it into a system that works like a gentle, permanent “nagging mom,” reminding you to stop procrastinating and take care of your responsibilities.

b. System outcomes (profits to the end user)

What the user actually gets out of this system:

- They always know how full their hamper is without checking.
- They get reminded when it’s the best time to wash clothes.
- They know exactly when the washer finishes instead of forgetting laundry inside for hours.
- They can monitor everything from a cloud dashboard anywhere.
- The whole process becomes smoother, less annoying, and less forgettable.
- It reduces pile-ups, smell, and the “do I have clean clothes?” moment.

Basically, the system acts like a personal assistant for laundry.

c. Technical description

The system runs on two ESP32 microcontroller nodes:

Node A – Hamper Level Sensor

- Uses an ultrasonic distance sensor to measure how high the clothes are.
- Converts the distance into a “percent full” value.
- Publishes these readings to the MQTT cloud broker every few seconds.

Node B – Washer Activity Sensor

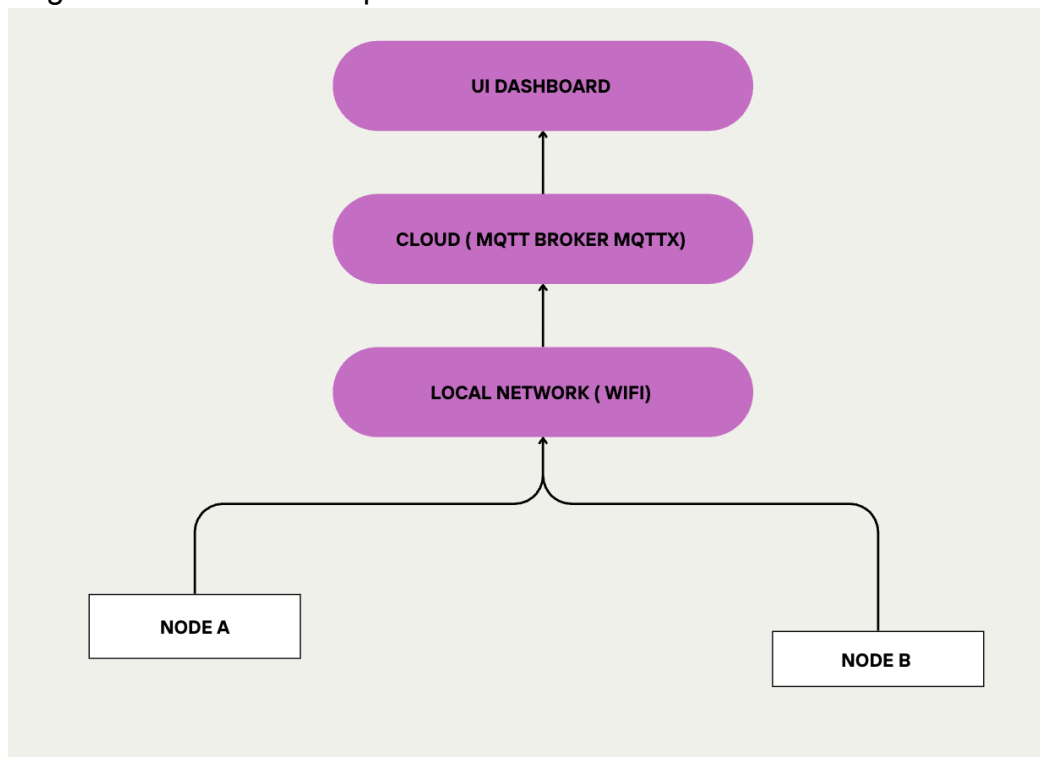
- Uses an Adafruit LIS3DH accelerometer to read vibration levels.
- Determines when the washer starts and stops based on movement patterns.
- Publishes “Washer Started” and “Washer Finished” messages to MQTT.

Both nodes connect through WiFi local network, send their data to the EMQX cloud MQTT broker, and the dashboard visualizes everything using Node-RED.

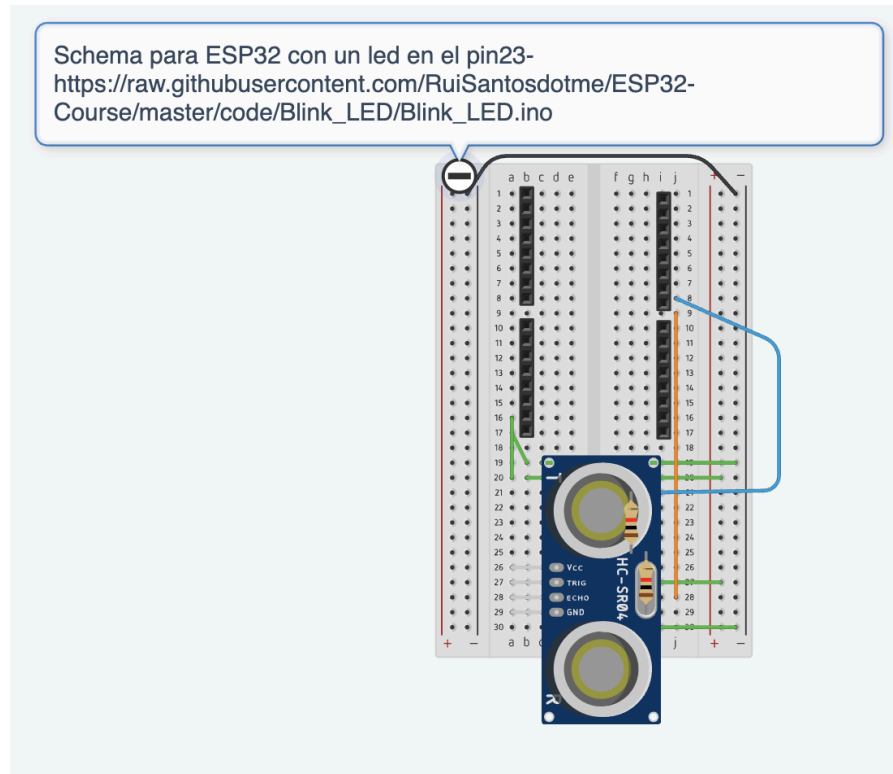
The dashboard updates in real time and gives the user a clean gauge for hamper level plus washer status text.

d. Diagrams

diagrams and their descriptions



Overall System flow with Node A and B connecting to wifi which sends signals through the cloud to display on the Dashboard.



Tinkercad diagram of NODE A

e. List of sensors

Vendor	Model	Comment
Generic	HC-SR04	Ultrasonic Distance Sensor: Used for measuring hamper fullness.
AdaFruit	LIS3DH	Accelerometer: Detects washer vibration and movement.

f. List of nodes

text

Vendor	Model	Comment
SparkFun	ESP32 Thing	Reads hamper height and publishes to MQTT.
Generic	ESP32 DEV Module	Reads washer vibration and publishes events.

g. List of other hardware components

text

Vendor	Model	Comment
Generic	Jumper Wires	Connections between sensors and ESP32.
Generic	Breadboard	Testing setup for both nodes.

Generic	Resistors	Used to create voltage Divider for 5v sparkfun for ultrasonic sensor two 1K Ohm resistors
---------	-----------	---

h. Selected cloud service

EMQX Cloud Free Tier

- Used as the MQTT broker.
- Accessible from anywhere.
- Supports clean topic organization:
 - hamper/level
 - washer/start
 - washer/finish

2. Project operation

a. Describe how your project works

Node A (Hamper Sensor):

- The ultrasonic sensor fires a pulse downward.
- The reflected time calculates distance.
- Distance is converted to "how full the hamper is."
- Publishes values like 34.7, 18.2, etc. to hamper/level.

Node B (Washer Sensor):

- The accelerometer constantly reads vibration levels.
- If shaking > threshold → washer running.
- If shaking stops for 5 seconds → washer finished.
- Publishes messages to:
 - washer/start
 - washer/finish

Dashboard Operation:

- Subscribes to all topics.
- Converts raw numbers into a gauge percentage.
- Displays washer status in real time.

b. Describe what's the data flow in your project

text

- i. specify publisher(s)
 - a. Node A publishes: hamper/level
 - b. Node B publishes: washer/start, washer/finish
- ii. specify subscriber(s)
 - a. Node-RED dashboard subscribes to all topics.

- iii. specify the location and type of MQTT broker
 - a. EMQX Cloud
 - b. Public broker address (w/ username & password)
- iv. specify the hierarchy / list of MQTT channels used
 1. hamper/level
 2. washer/start
 3. washer/finish

c. Describe cloud operation, including what's the data processing mechanism

On the cloud side, EMQX simply handles the MQTT traffic.
Node-RED does the processing:

- Takes hamper/level readings
- Converts them to percentages using a function node
- Updates the dashboard gauge

For the washer:

- Receives text messages ("Washer Started" / "Washer Finished")
- Updates a status widget in the dashboard

3. Screenshots / pictures

- a. Publisher(s) operation (data being sent to MQTT broker)**


```

1 //----- WIFI -----
2 #include <PubSubClient.h>
3
4 // ----- WIFI CREDENTIALS -----
5 const char* ssid = "my qf";
6 const char* password = "Password25";
7
8 // ----- MQTT BROKER -----
9 const char* mqtt_server = "192.168.0.12";
10 const char* topic_level = "hamper/level";
11

```

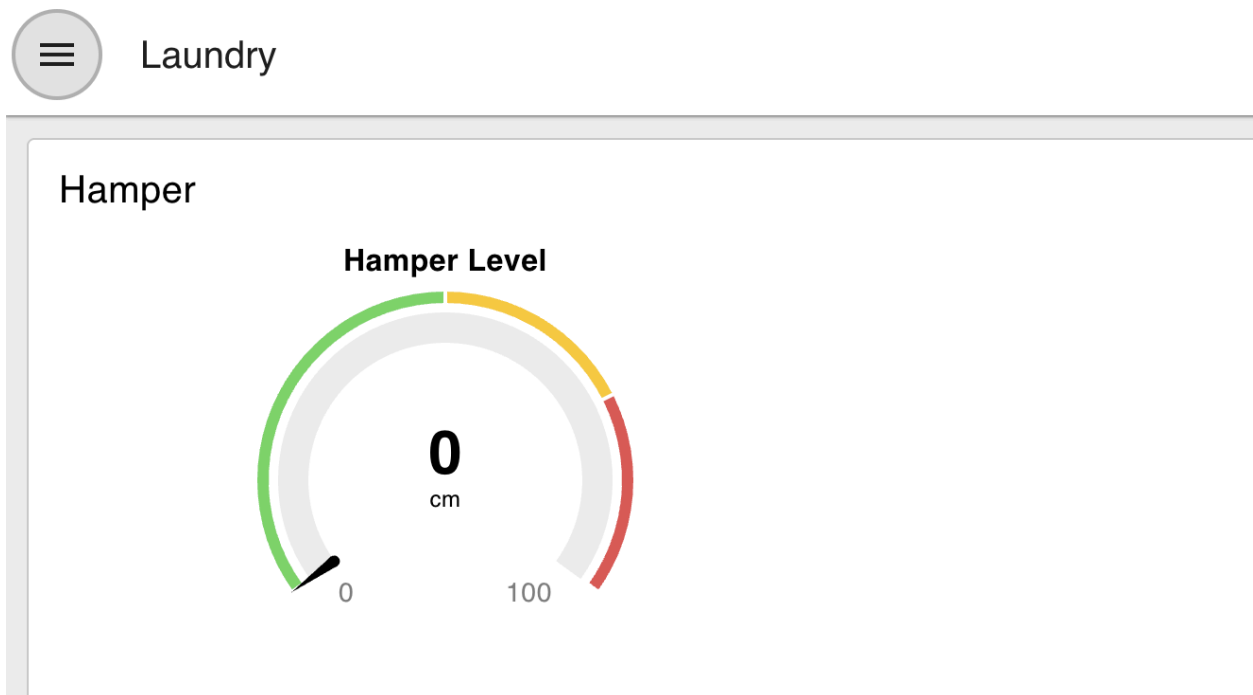
Message (Enter to send message to 'SparkFun ESP32 Thing' on 'COM7')

Node A Working

```
WiFi connected!  
[REDACTED]  
LIS3DH ready!  
Connecting to MQTT...connected!  
EVENT - Washer Started  
EVENT - Washer Finished
```

Node B Working (Block bar is to stop from showing my home IP address, it was used for checking connection to correct network)

b. Subscriber(s) operation (data being received from MQTT broker)



Red UI Hamper level Gauge

c. Log of the channels activity (if broker permits)

EMQX_Local

+ New Subscription

hamper/level

QoS 0

Washer/Start

QoS 0

Washer/Finish

QoS 0

Plaintext

AllReceivedPublished

31.7

2025-12-08 05:15:15:713

Topic: hamper/level QoS: 0

31.6

2025-12-08 05:15:17:735

Topic: hamper/level QoS: 0

31.6

2025-12-08 05:15:19:767

Topic: hamper/level QoS: 0

31.6

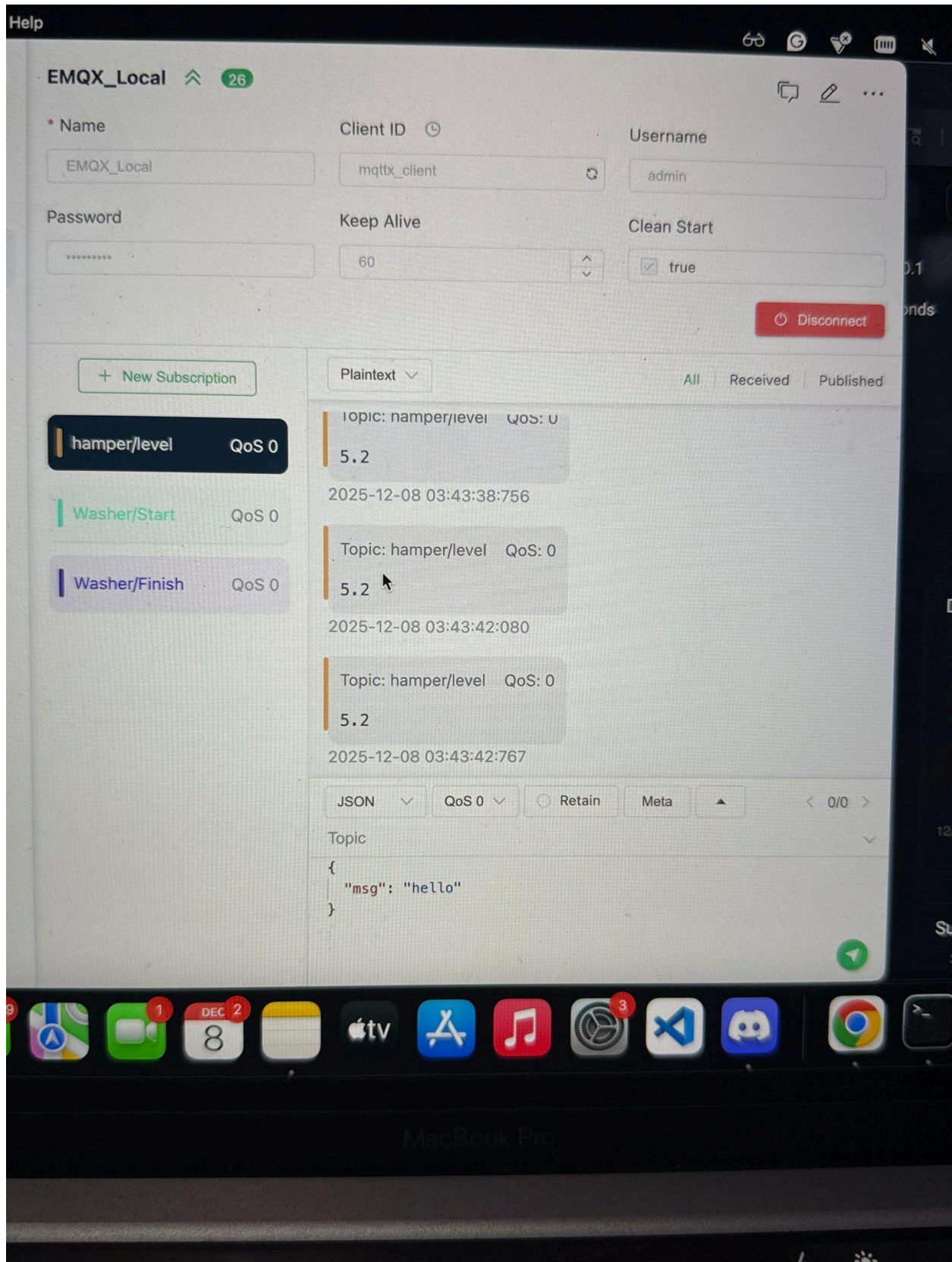
2025-12-08 05:15:22:932

Topic: hamper/level QoS: 0

31.7

2025-12-08 05:15:23:694

MQTTX Showing a semi empty hamster

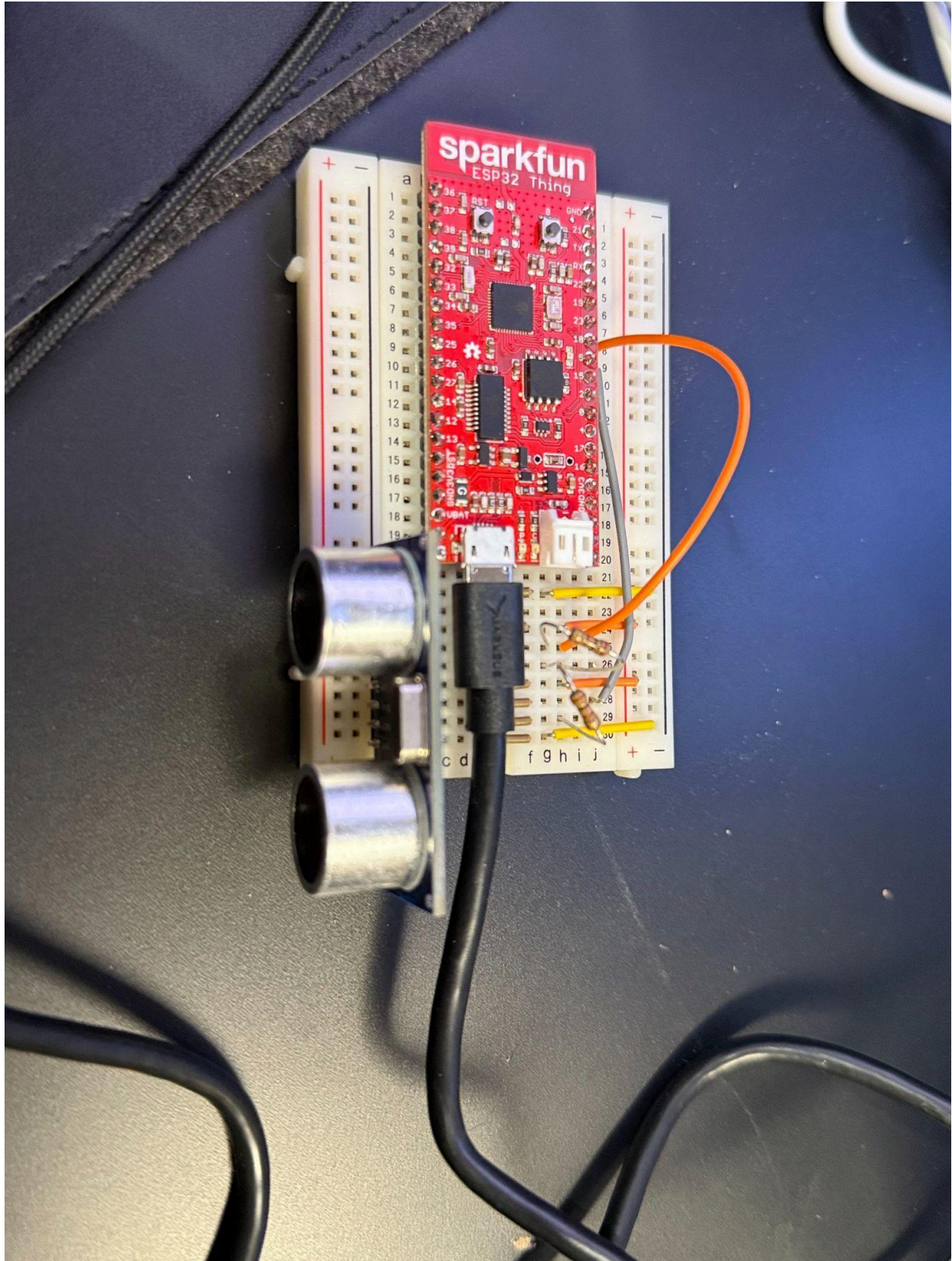


MQTTX Showing Full hamper

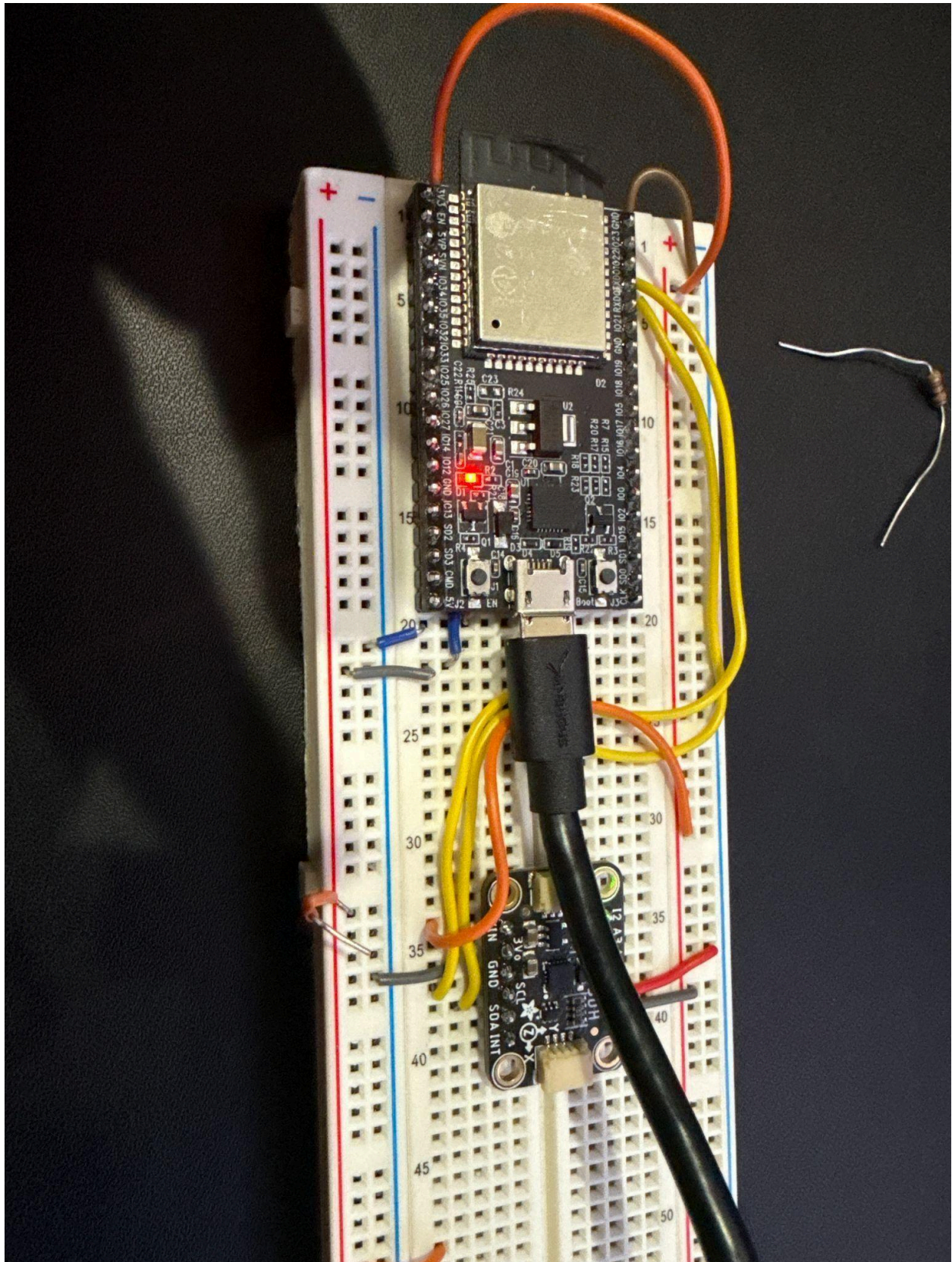
d. Picture(s) of wired circuit



Node A actually attached to Hamper, Also realized that attaching lower give a better reading but unfortunately wasn't able to get a picture of that.



Wiring for Node A.



Wiring for Node B.

4. Describe technical challenges you had and how you solved them

Working on this project came with way more challenges than I expected, especially because it was the first time working with MQTT. The first big issue was the hotel WiFi. Since it was working at home I expected that all I would need is my laptop and wifi for it to work again. The Wifi like most hotel Wifi's have an external website for identification which would not work for the ESP32 obviously. Even when I switched to my phone hotspot, it basically refused to let the ESP32 talk to anything. The MQTT broker wouldn't connect, Node-RED couldn't see my device, and nothing was showing up. I finally realized the WiFi had device isolation and so there was no workaround unfortunately so I traveled with my kits for no reason.

MQTT also gave me headaches. At first the ESP32 kept spamming "failed, rc = -2," and I thought the code was broken. Turns out I had messed up the broker settings and forgot to add the username/password correctly for EMQX. After fixing that, the connections finally held steady. Then Node-RED decided to be difficult. The gauge on the dashboard was stuck at 0 no matter what I did, even though I could literally see the sensor publishing real readings in MQTTX. The problem was that the ultrasonic sensor was sending the value as a string, and the gauge needed a clean number. I added one line in the function node to convert it, and suddenly everything worked.

The hardware also fought me a bit. The ultrasonic sensor struggled because clothes don't reflect sound well, so sometimes it acted blind. I had to put something flat on top of the laundry to make the readings stable. The accelerometer node also had trouble at first because the washer I tested on doesn't shake much. I had to tweak the sensitivity until it consistently detected when the washing machine was actually running.

Overall, the process felt like a long chain of "fix one thing, find another problem", but after enough trial and error, I got both nodes publishing real data, the dashboard updating, and the whole system finally felt like a real IoT project.

5. Code listings for each node and cloud codes

Node A – Hamper Sensor (ESP32)

```
#include <WiFi.h>
#include <PubSubClient.h>

// WIFI CREDENTIALS

const char* ssid = "my qf";
const char* password = "Password25";
```



```

// MQTT BROKER
const char* mqtt_server = "192.168.0.12";    // <--- FIXED
const char* topic_level = "hamper/level";

WiFiClient espClient;
PubSubClient client(espClient);

// ULTRASONIC PINS
#define TRIG 5
#define ECHO 18

long duration;
float distance;

void setup_wifi() {
  Serial.print("Connecting to WiFi...");
  Serial.println(ssid);

  WiFi.begin(ssid, password);
  int tries = 0;

  while (WiFi.status() != WL_CONNECTED) {
    delay(300);
    Serial.print(".");
    tries++;

    if (tries > 40) {
      Serial.println("\nFAILED TO CONECT");
      return;
    }
  }
  Serial.println("\nWiFi connected!");
  Serial.print("ESP32 IP Address: ");
  Serial.println(WiFi.localIP());
  Serial.print("ESP32 Wifi RSSI: ");
  Serial.println(WiFi.RSSI());
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Connecting to MQTT...");
    if (client.connect("NodeA_Hamper", "admin", "Butface11")) {
      Serial.println("connected!");
    } else {
      Serial.print("failed, rc=");
      Serial.println(client.state());
    }
  }
}

```

```

        delay(2000);
    }
}

void setup() {
    Serial.begin(115200);
    setup_wifi();

    client.setServer(mqtt_server, 1883);

    pinMode(TRIG, OUTPUT);
    pinMode(ECHO, INPUT);
}

float readUltrasonic() {
    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG, LOW);

    duration = pulseIn(ECHO, HIGH);
    distance = duration * 0.034 / 2; // cm

    return distance;
}

void loop() {
    if (!client.connected()) reconnect();
    client.loop();

    float level = readUltrasonic();

    char msg[20];
    sprintf(msg, "%.1f", level);

    client.publish(topic_level, msg);

    Serial.print("Published hamper level: ");
    Serial.println(msg);

    delay(2000);
}

```

Node B – Washer Accelerometer Node (ESP32)

```
#include <Wire.h>
#include <WiFi.h>
#include <PubSubClient.h>
#include "SparkFunLIS3DH.h"

LIS3DH myIMU;

// WIFI
const char* ssid = "my qf";
const char* password = "Password25";

// MQTT
const char* mqtt_server = "192.168.0.12";
const int mqtt_port = 1883;
const char* mqtt_user = "admin";
const char* mqtt_pass = "Butface11";

// MQTT Topics
const char* topic_start = "Washer/Start";
const char* topic_finish = "Washer/Finish";

WiFiClient espClient;
PubSubClient client(espClient);

// Washer state detection
bool washerRunning = false;
unsigned long stillnessStart = 0;

void setup_wifi() {
  Serial.print("Connecting to WiFi...");
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected!");
  Serial.print("ESP32 IP: ");
  Serial.println(WiFi.localIP());
}
```



```

void reconnect() {
  while (!client.connected()) {
    Serial.print("Connecting to MQTT...");
    if (client.connect("NodeB_Washer", mqtt_user, mqtt_pass)) {
      Serial.println("connected!");
    } else {
      Serial.print("failed, rc=");
      Serial.println(client.state());
      delay(1500);
    }
  }
}

```

```

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);

  setup_wifi();

  client.setServer(mqtt_server, mqtt_port);

  if (myIMU.begin() == false) {
    Serial.println("LIS3DH not detected!");
    while (1);
  }

  Serial.println("LIS3DH ready!");
}

```

```

void loop() {
  if (!client.connected()) reconnect();
  client.loop();

  float x = myIMU.readFloatAccelX();
  float y = myIMU.readFloatAccelY();
  float z = myIMU.readFloatAccelZ();

  float magnitude = sqrt(x*x + y*y + z*z);

  // Washer motion threshold
  if (magnitude > 1.20) {
    if (!washerRunning) {
      washerRunning = true;
      Serial.println("EVENT → Washer Started");
      client.publish(topic_start, "Washer Started");
    }
  }
}

```

```
    stillnessStart = millis();  
}  
  
// Washer finish detection when still for 5 seconds  
if (washerRunning && millis() - stillnessStart > 5000) {  
    washerRunning = false;  
    Serial.println("EVENT → Washer Finished");  
    client.publish(topic_finish, "Washer Finished");  
}  
  
delay(200);  
}
```